

The Context Vault

A starter kit for the file-based memory layer underneath your AI

This is the actual structure I use to give an AI persistent memory across everything I work on — scrubbed of anything specific to me, ready for you to copy. It's a folder of plain Markdown files. No database, no framework, no platform. Steal all of it.

I didn't invent any of the parts; I assembled them into something that fits how I work. Where the ideas come from is at the end — credit where it's due.

How to use this: read it once top to bottom, then create the folder, paste in the templates, and start dropping your own context in. The whole thing takes an afternoon to stand up and pays for itself the first week.

Prefer to have your AI build it with you? This kit includes a companion **builder** — a Markdown file written for your AI to run, not for you to read. Grab it from the library alongside this guide, upload or paste it into your assistant, and say “run this.” It interviews you about your projects and generates the whole structure, filled in for your actual work — writing the files to disk if it can, or handing them to you to paste if it can't. This guide is the map; the builder walks the route with you.

The idea, in one paragraph

Every LLM you talk to starts from zero. The models that seem to “remember” you do it because something underneath them holds context across sessions and loads it back in. That something doesn't have to be a product — it can be a folder of files you own. The skill isn't writing a clever prompt; it's putting the right context in front of the model and leaving the rest out. (That skill has a name now — context engineering — but the practice is older than the label.) The catch: the context an AI needs is yours, and you're the only one who can organize it. The upside: once it's done, it's done. It's portable, it survives model changes, and it works with any AI that can read a file.

The three layers

The whole discipline is keeping three kinds of things separate, because they follow different rules.

Layer	Folder	Mutability	Holds
1 · Raw	01-sources/	add-only — never edit	Original inputs: chat exports, transcripts, articles, documents
2 · Synthesized	02-synthesized/	mutable (git-versioned)	Working memory: distillations, current-state anchors, briefings
3 · Wiki	10-<domain>/	curated (frontmatter + links)	Compounding knowledge: project pages, decisions, concepts

Why the separation matters. Because Layer 1 is truly immutable, your raw record stays trustworthy forever — you can always trace back to what you originally saw. All edits land in Layer 2, where git keeps the history. Only reviewed, durable knowledge graduates to Layer 3. That clean provenance is what lets an AI read widely without you ever losing track of what's ground truth versus working draft.

The most common failure is skipping straight to a perfect Layer 3 without building the intake habit (Layer 1) and the synthesis habit (Layer 2). It falls apart fast. Start at the bottom.

The folder map

A starter shape. Adapt the names; keep the structure.

```
~/vault/
  AGENTS.md           # the entry file — every agent reads this first
  01-sources/         # raw inputs — add-only, never edit
  02-synthesized/     # working memory — mutable, the AI reads this most
  10-<your-domain>/  # the wiki — one tree per area of your life/work
    <project>/        # a project's pages, decisions, state
  30-concepts/       # shared ideas/frameworks that cross projects
  _meta/             # the conventions themselves (schema, not content)
    decisions/       # ADRs — your decision records
```

Stand it up:

```
mkdir -p ~/vault/{01-sources,02-synthesized,30-concepts}
mkdir -p ~/vault/_meta/decisions
mkdir -p ~/vault/10-<your-domain>/<project>
touch ~/vault/AGENTS.md # the entry file — fill it from the template below
cd ~/vault && git init && printf "*.env\n_secrets/\n" > .gitignore
git add -A && git commit -m "vault: skeleton"
```

Naming conventions (adopt verbatim)

- **Time-ordered artifacts:** `<topic>-YYYY-MM-DD.md` — sorts automatically, collisions are obvious.
- **Decision records:** `ADR-00N-<slug>.md`, numbered, with the status in the title.
- **Write paths as `~/`-relative inside files** — never `/Users/you/...`. The day you sync to a second machine with a different username, you'll be glad.

Frontmatter (a few fields, on every file that matters)

Put a small block at the top of synthesized and wiki files. It's how you (and the AI) know what something is and whether it's current.

```
---
type: anchor # anchor | source | wiki | decision
updated: 2026-06-15 # the single most useful field — is this still current?
tags: [project-x, current-state]
status: active # active | archived
---
```

The `updated:` date does more work than anything else. Stale-but-confident context is worse than no context; the date is how you and the model both know whether to trust a file.

The anchor — the one document that matters most

An **anchor** is a short, ground-truth statement of where a project actually stands right now. Not its history, not everything about it — the current state. When the AI loads a project, it reads the anchor first; everything else is detail. Keep one per active project in `02-synthesized/`.

Copy this template:

```
---
type: anchor
updated: 2026-06-15
tags: [<project>, current-state]
status: active
---

# <Project> — anchor

## Current state (as of 2026-06-15)
- Phase: <e.g. active build, on hold, winding down>
- Status: <two lines on where things actually stand>
- Blockers: <anything in the way, or "none">

## Decided
- <decision>: <the conclusion, one line> (ref: ADR-003)
- <decision>: <the conclusion, one line>

## Not yet decided
- <open question>
- <open question>

## Context the AI must hold
- <constraint — e.g. "must work offline">
- <constraint — e.g. "auth uses X, not Y">
```

The discipline: when a project changes, **update its anchor**. Don't leave the old state sitting next to the new one hoping you'll remember which is which. One edit, one line, done.

The bridge: one file every agent reads first

This is the piece that ties a folder of files to an AI actually using it well.

Instead of re-explaining your system at the start of every session, you put one file at the root — `AGENTS.md` — that any agent reads first to orient itself: what the vault is, where the current state lives, how to find what it needs, what it must never touch. Write it once; every session boots from it.

This one isn't mine either. In the coding-agent world it's converged into an open standard — `AGENTS.md`, read by most agents and tools (Claude uses its own `CLAUDE.md` for the same job; you can point that at this file in one line). One file, every agent. Here it does the same thing for a knowledge vault instead of a codebase.

A starter to drop in at the root:

```
# AGENTS.md — read this first

You're working inside my context vault: a folder of Markdown files that holds what
I'm working on. Orient yourself here before doing anything.

## What this is — three layers, different rules
- 01-sources/      raw inputs — never edit
- 02-synthesized/  working memory: anchors (current state) + summaries — edit freely
- 10-<domain>/     the wiki: durable pages, decisions, concepts

## How to start any task
1. Find the project's anchor in 02-synthesized/ and read it first — that's the current
   state.
2. Read only the wiki pages relevant to the task. You don't need everything.
3. If anything I ask contradicts the anchor, flag it instead of guessing.

## Conventions
- Files: <topic>-YYYY-MM-DD.md. The `updated:` field tells you if a file is still
  current.
- Paths are ~/-relative. Decisions live in _meta/decisions/ as ADR-00N-*.md.

## Boundaries
- Never edit anything in 01-sources/.
- Don't invent state — if the anchor doesn't say it, it isn't decided.
- <add your own: which areas an agent may read, what it must never touch>
```

The moment this file exists, you stop being the integration layer. The agent self-ori-ents; you hand it the task.

The prompts that load the work

With `AGENTS.md` handling orientation, these aren't for re-explaining the system — they're for loading the specific work.

1 · The session opener — point at the anchor, then give the goal.

```
We're working on <project>. Read its anchor in 02-synthesized/ for current state.
Today's goal: <the one specific thing>.
Work from the anchor; if I contradict it, flag that instead of guessing.
```

2 · The state update — one line at session end.

We changed <X> today. Update the "Current state" block of <project>'s anchor to match, keep it to five lines, and set updated: to today's date.

3 · The explicit load — the context has to be present. Pointing at "a file somewhere" does nothing. The anchor and any pages have to be in the session — read by the agent, pasted, or uploaded. If the model can't see it, it isn't loaded.

Why this is portable (the part that matters in a year — and across machines)

The vault is just files. Plain-text Markdown. They sync with whatever syncs files — git, Syncthing, Dropbox. The AI that reads them can be anything with file access.

And because the orientation lives in `AGENTS.md` — a file an open ecosystem of agents already reads — any agent can boot from it: a different model, a different framework, even an agent running on a different machine. Same entry point, same context, same behavior. That's the part that quietly turns a personal memory store into something bigger: one vault, many agents, consistent across tools and machines. The day you want to hand a task to a second agent — or run one on an always-on box while you work on your laptop — the groundwork is already laid. (Its own piece, later.)

The bet underneath all of it: models improve fast, tools come and go. If your memory layer lives inside one platform, you're wagering on that platform. If it's files plus a standard entry point, you move freely — point the new thing at the same folder and keep going. A year from now the tools will have changed; the folder will be the same folder.

The one question to ask once it's built

When this is running, the AI remembers across everything you work on. That's the point — and it raises a question worth answering on purpose:

Should the agent working on Project A be able to read Project B's context?

For most people, on one machine, the real risk is just accidental bleed — the AI reaching for the wrong project's context because it happened to be loaded. For anyone juggling clients under NDA, or walling work off from personal projects, it matters more. The answer isn't always "no" — but it should be a decision, not an accident. (Its own piece, too.)

Credits & sources — where this comes from

None of the parts are mine:

- The **layered structure** is personal knowledge management pointed at an AI instead of at you — the second brain / progressive-summarization idea (Tiago Forte) and the older Zettelkasten lineage.
- "**Load the right context, not all of it**" is what Andrej Karpathy and others started calling context engineering in 2025.
- The **single entry-file** pattern is the `AGENTS.md` / `CLAUDE.md` convention from the coding-agent world (`AGENTS.md` is now an open standard many tools read).

The only original part is the assembly — fitting the pieces to how I actually operate. That's also the part you can't copy from anyone: you have to make it yours. Steal the structure; build your own version.

This kit is free — copy it, adapt it, share it. It's the working structure, not a product.